

You may wonder what this drivel has to do with Assembly, so here goes: Programming Assembly is like sticking individual pins on the cylinder that will later turn the gears that produce music.

Before there was assembly, computers needed to be programmed by speaking their language directly with individual 0s and 1s. For instance the sequence 10110111 00001011 might instruct the computer to store the value 11 in a particular part of its memory.

For instance, in the above example, the first 4 bits of the instruction (1011) might stand for “move value”, the second set of 4 bits (0111) might be the specific location in memory where the value is to be moved, and finally the last 8 bits (00001011) are just the binary encoding of the decimal number 11. The first part of the instruction is called the opcode, or operation code. It's like telling the computer which note to play.

What Assembly does is simply give this machine-speak a human face by translating what the computer is doing into instructions that are somewhat more understandable.

An Assembly representation of this instruction might be:

- `mov ax, 11`

Here ‘ax’ is the name of a memory register in your processor. In a higher-level programming language you might write this as:

- `var ax = 11`

At some level though, the above assembly instruction is the kind of thing that the CPU is finally executing.

If you want to, you can take a piece of assembly code, and look up its equivalent binary machine instruction (although usually represented in hex) and you will see exactly what the computer sees when that instruction is run.

Of course, the examples we have given above are mostly made up. Those aren't the exact opcodes and binary representations for these instructions. In fact these instructions aren't universal, and neither are their opcodes. There is no single Assembly language syntax, since Assembly is closely tied to the architecture of the machine on which it is intended to be run. As we mentioned before, it is essentially machine language put into English letters and symbols.

A knowledge of Assembly therefore requires a knowledge of the machine on which it will run, unlike C or Python code which just needs to be compiled on different platforms to get it running there.

This is part of the reason why higher-level languages have almost entirely replaced Assembly code. Assembly language is daunting and there is no getting around that fact.